

Elimination orderings with CliqueTrees

Simon Frost

Table of contents

1 Overview	1
2 Setup	2
3 A chain and a grid	2
4 Keeping the query variables	3
5 Timing on a random network	3
6 Summary	5
7 References	5

1 Overview

The cost of variable elimination is governed by the *order* in which variables are summed out: the largest intermediate factor has one axis per variable in the largest clique of the graph induced by the order, so its size is exponential in the order's width. Finding the optimal order (whose width is the treewidth) is NP-hard, and `BayesianNetworkInference.jl` delegates the heuristics to [CliqueTrees.jl](#) (Samuelson and Fairbanks 2026):

strategy	CliqueTrees algorithm	notes
<code>MinFill()</code>	<code>MF()</code>	greedy minimum fill; the default
<code>MinDegree()</code>	<code>MMD()</code>	multiple minimum degree
<code>AMDOrder()</code>	<code>AMD()</code>	approximate minimum degree (SuiteSparse)
<code>ExactTreewidth()</code>	<code>BT()</code>	exact Bouchitte-Todinca; exponential
<code>UserOrder(vars)</code>	the given permutation	validated

2 Setup

```
using BayesianNetworkInference
using FiniteKernels
using Random
```

3 A chain and a grid

Pairwise factors on a chain have treewidth 1; on a $k \times k$ grid the treewidth is k . Every strategy finds the chain's optimum; on the grid the heuristics are compared with the exact value.

```
ax(v) = FiniteAxis(v, [:a, :b])
pair(u, v) = Factor([ax(u), ax(v)], rand(MersenneTwister(hash((u, v))), 2, 2))

chain = FactorGraph([pair(Symbol(:V, i), Symbol(:V, i + 1)) for i in 1:9])
strategies = [MinFill(), MinDegree(), AMDOrder(), ExactTreewidth()]
[typeof(s) => treewidth(chain, s) for s in strategies]
```

```
4-element Vector{Pair{DataType, Int64}}:
  MinFill => 1
  MinDegree => 1
  AMDOrder => 1
  ExactTreewidth => 1
```

```
gv(i, j) = Symbol(:G, i, j)
grid = Factor{Float64}[]
for i in 1:4, j in 1:4
    i < 4 && push!(grid, pair(gv(i, j), gv(i + 1, j)))
    j < 4 && push!(grid, pair(gv(i, j), gv(i, j + 1)))
end
grid_fg = FactorGraph(grid)
[typeof(s) => treewidth(grid_fg, s) for s in strategies]
```

```
4-element Vector{Pair{DataType, Int64}}:
  MinFill => 4
  MinDegree => 4
  AMDOrder => 4
  ExactTreewidth => 4
```

A user-supplied order is checked and its width reported too. Eliminating the grid row by row is optimal; eliminating the diagonal first is not.

```
rows = [gv(i, j) for i in 1:4 for j in 1:4]
diagonal_first = vcat([gv(i, i) for i in 1:4], setdiff(rows, [gv(i, i) for i in 1:4]))
treewidth(grid_fg, UserOrder(rows)), treewidth(grid_fg, UserOrder(diagonal_first))
```

(4, 5)

4 Keeping the query variables

Variable elimination sums out everything except the query. `elimination_order` takes a keep list and forces those vertices to the end of the CliqueTrees permutation with `CompositeRotations(keep, alg)`, then drops them, so that the fill-reducing heuristic still sees the whole graph.

```
keep = [gv(1, 1), gv(4, 4)]
order = elimination_order(grid_fg, MinFill()); keep=keep)
length(order), any(in(keep), order)
```

(14, false)

`variable_elimination` reports the order it used and the width it induced:

```
post, diag = variable_elimination(grid_fg, keep)
diag.order == order, diag.treewidth, diag.max_factor_size
```

(true, 5, 64)

```
post ≈ brute_force_marginal(grid_fg, keep)
```

true

5 Timing on a random network

A random Bayesian network with 60 binary variables, each with up to three parents among the previous eight, is large enough for the ordering to matter and small enough to eliminate in milliseconds. `ExactTreewidth()` is excluded here: the Bouchitte-Todinca algorithm is exponential and meant for small graphs and reference values.

```
function random_network(rng, n; window=8, maxparents=3)
    kernels = Pair{Symbol,FiniteKernel}[]
    parents = Dict{Symbol,Vector{Symbol}}{]()
    for i in 1:n
        v = Symbol(:X, i)
        cands = [Symbol(:X, j) for j in max(1, i - window):(i - 1)]
        ps = shuffle(rng, cands)[1:min(rand(rng, 0:maxparents), length(cands))]
        parents[v] = ps
        dom = FiniteSpace(FiniteAxis[ax(p) for p in ps])
        push!(kernels, v ⇒ random_kernel(rng, dom, FiniteSpace(ax(v))))
    end
```

```

    return kernels, parents
end

```

```

rng = MersenneTwister(2026)
kernels, parents = random_network(rng, 60)
big = FactorGraph([Factor(k, parents[v], v) for (v, k) in kernels])
query = [:X60]

```

```

1-element Vector{Symbol}:
 :X60

```

```

results = map([MinFill(), MinDegree(), AMDOrder()]) do s
    variable_elimination(big, query; order=s) # warm up compilation
    t = @elapsed post, d = variable_elimination(big, query; order=s)
    (strategy=typeof(s), width=d.treewidth, max_factor_size=d.max_factor_size,
     multiplications=d.n_multiplications, seconds=round(t; digits=4))
end
foreach(println, results)
(julia = string(VERSION), cpu = Sys.CPU_NAME, threads = Threads.nthreads(),
 repetitions = 1, statistic = "single run after warm-up")

```

```

(strategy = MinFill, width = 4, max_factor_size = 32, multiplications = 59, seconds
= 0.0007)
(strategy = MinDegree, width = 4, max_factor_size = 32, multiplications = 59,
seconds = 0.0004)
(strategy = AMDOrder, width = 4, max_factor_size = 32, multiplications = 59, seconds
= 0.0004)

```

```

(julia = "1.12.7", cpu = "apple-m1", threads = 1, repetitions = 1, statistic =
"single run after warm-up")

```

These are measurements of this execution, not portable timing guarantees. Repeated measurements would be needed for a performance comparison.

A poor order shows up immediately: eliminating in reverse topological order builds large intermediate factors.

```

reverse_order = UserOrder(reverse([Symbol(:X, i) for i in 1:59]))
t = @elapsed post_rev, d_rev = variable_elimination(big, query; order=reverse_order)
(width=d_rev.treewidth, max_factor_size=d_rev.max_factor_size, seconds=round(t;
digits=4))

```

```

(width = 7, max_factor_size = 256, seconds = 0.6518)

```

All orders give the same posterior:

```

posts = [variable_elimination(big, query; order=s)[1] for s in [MinFill(),
MinDegree(), AMDOrder(), reverse_order]]
all(p ≈ posts[1] for p in posts)

```

true

The exact treewidth is affordable on a 20-variable slice of the same construction and lets us see how far the heuristics are from optimal:

```
small_kernels, small_parents = random_network(MersenneTwister(7), 20)
small = FactorGraph([Factor(k, small_parents[v], v) for (v, k) in small_kernels])
[typeof(s) ⇒ treewidth(small, s) for s in strategies]
```

```
4-element Vector{Pair{DataType, Int64}}:
  MinFill => 6
  MinDegree => 6
  AMDOrder => 6
  ExactTreewidth => 6
```

6 Summary

The elimination order decides the cost of variable elimination, and the width of the order – the size of the largest intermediate factor – is the quantity to watch; the heuristics of `CliqueTrees.jl` (Samuelson and Fairbanks 2026) find near-optimal orders in negligible time, while the exact Bouchitte-Todinca solver is affordable only on small graphs and is used here as the reference the heuristics are measured against. Every order gives the same posterior, so the choice is purely one of cost. The next vignette, *Sampling and Monte Carlo checks*, turns to the approximate side and uses ancestral sampling as an independent check on these exact answers.

7 References

Samuelson, Richard, and James Fairbanks. 2026. *CliqueTrees.jl: Tree Decompositions and Elimination Orderings in Julia*. <https://github.com/AlgebraicJulia/CliqueTrees.jl>.